Agente de pedidos para un emprendimiento de comidas

Proyecto final: automatización con n8n, AI Agent y RAG

Automatización de la atención de pedidos de un emprendimiento de comidas caseras con menú semanal. Los pedidos llegan por un Google Form, un agente de IA los revisa contra el menú y las políticas del negocio, y decide si confirmarlos, rechazarlos, pedir más datos o derivarlos al dueño.

Stack: n8n Cloud, Google Sheets / Forms / Gmail / Docs, Supabase (pgvector) y Gemini.

Archivos del repositorio

- `workflow-agente-pedidos.json`: workflow principal.
- `workflow-carga-rag.json`: workflow que carga el documento de políticas en Supabase (se corre una sola vez).
- `workflow-alertas-error.json`: workflow con el Error Trigger.
- `/capturas`: capturas del canvas y de las pruebas.

1. Caso de negocio y ROI

Proceso que se automatiza

Hoy cada pedido que llega por formulario se atiende a mano: leo el pedido, miro el menú del día, verifico el stock, reviso si el cliente mencionó alguna alergia, me fijo si la cantidad entra dentro de lo permitido, escribo la respuesta por mail y anoto el estado en la planilla. Es un trabajo repetitivo y con errores fáciles (olvidarme de descontar stock, no ver una observación sobre celiaquía, contestar tarde).

Cálculo estimado

Como el emprendimiento todavía no se lanzó, los números son estimaciones.

Dato	Valor
Pedidos por día (lunes a viernes)	25
Días de atención por mes	21
Pedidos por mes	525
Tiempo manual por pedido	4 min
Valor de mi hora de trabajo	$5.000

Costo manual: 525 pedidos × 4 min = 2.100 min = 35 horas por mes, es decir $175.000 por mes.

Con el sistema: se estima que cerca del 20% de los pedidos (derivaciones) sigue necesitando revisión mía, a 2 minutos cada uno. Eso son unas 3,5 horas por mes. Se ahorran aproximadamente 31,5 horas, unos $157.500 por mes.

Costos del sistema:

- Implementación (única vez): unas 15 horas de armado y pruebas, $75.000.
- Mensual: n8n Cloud y uso de la API de Gemini, unos $35.000. Supabase y Google se usan en plan gratuito.

Resultado:

- Ahorro neto mensual: $157.500 − $35.000 = $122.500.

- Recuperación de la inversión: $75.000 / $122.500, alrededor de 0,6 meses (menos de tres semanas).

Por qué escala mejor que lo manual

Si los pedidos se duplican, el trabajo manual se duplica. Con el workflow el costo casi no cambia, porque cada pedido extra suma centavos de API. Además responde en un par de minutos, a cualquier hora, y aplica siempre las mismas reglas.

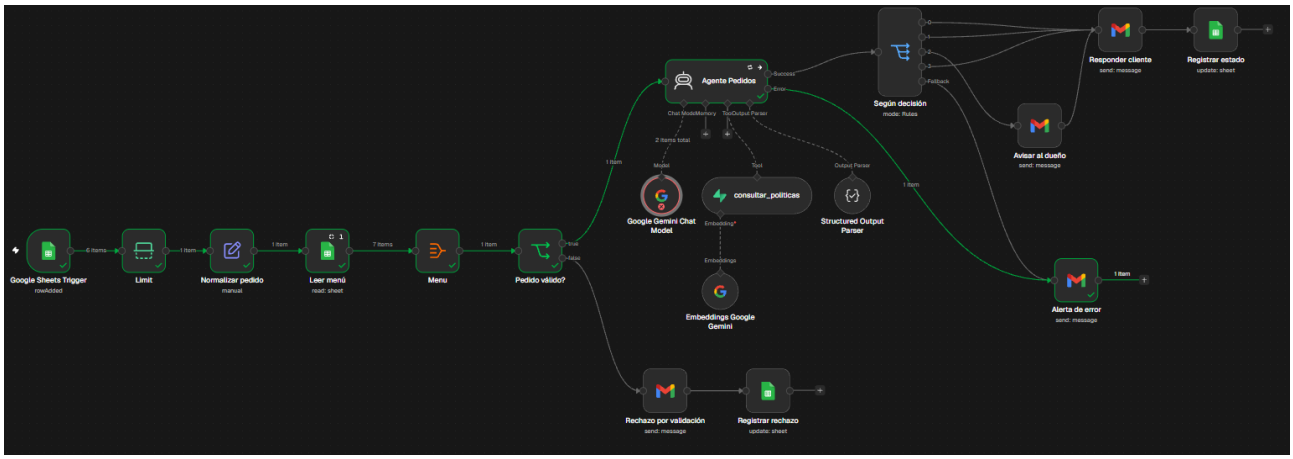
Límites del ROI

- **Volumen bajo:** el ahorro por pedido es de unos 3 minutos (≈ $267). Con el costo mensual de $35.000 hace falta llegar a unos 130 pedidos por mes (≈ 6 por día) para que convenga. Debajo de eso, atender a mano sale igual o más barato.
- **Muchos pedidos especiales:** si la mayoría de los pedidos fueran eventos, modificaciones o dietas particulares, casi todo se derivaría y el agente no ahorraría tiempo.
- **Menú o reglas que cambian todos los días:** habría que recargar el documento del RAG seguido y el mantenimiento le quitaría ventaja al sistema.

Por qué es buen candidato para RAG

Las reglas del negocio (horarios, zonas, máximo por pedido, alérgenos por plato, cancelaciones, pedidos especiales) son texto que cambia de tanto en tanto y que no entra cómodo en un prompt fijo. Con RAG el agente consulta solo el fragmento que necesita, y si cambia una política se actualiza el documento sin tocar el workflow.

2. Arquitectura del workflow



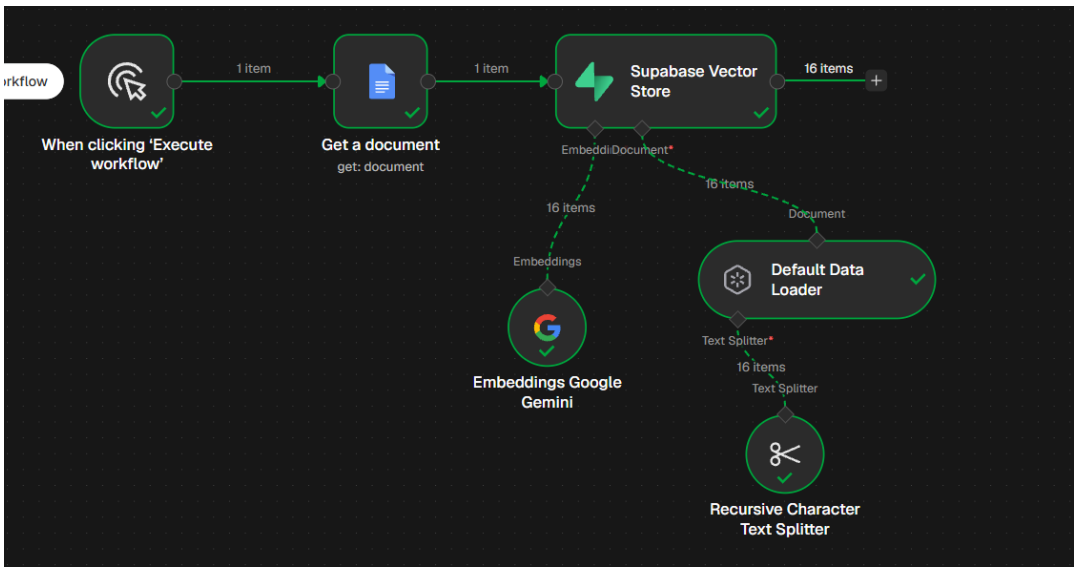
Canvas del workflow principal

Recorrido de un pedido:

1. **Google Sheets Trigger** (entrada 1): detecta cada fila nueva que genera el Google Form en la hoja `Pedidos`.
2. **Limit** (último ítem): evita que un test event con varias filas procese pedidos de más.
3. **Normalizar pedido** (Edit Fields): deja los campos con nombres simples (`nombre`, `email`, `dia`, `plato`, `cantidad`, `observaciones`, `marca`), porque las columnas del Form vienen con el texto completo de cada pregunta.
4. **Leer menú** (Google Sheets, entrada 2): trae el menú semanal con ingredientes, stock y precio. Tiene activado *Execute Once* para que no repita las filas.
5. **Menu** (Aggregate): junta las filas del menú en un solo ítem, para que el agente se ejecute una sola vez por pedido.

6. **Pedido válido?** (If): comprueba que el plato exista ese día en el menú (ignorando mayúsculas, tildes y espacios) y que la cantidad sea mayor a cero.
7. **Agente Pedidos** (AI Agent): decide qué hacer. Usa la herramienta `consultar_politicas` (RAG sobre Supabase) y devuelve la decisión con un Structured Output Parser.
8. **Según decisión** (Switch): reparte según `confirmar`, `rechazar`, `derivar` o `pedir_datos`, con una salida de fallback para valores inesperados.
9. **Responder cliente** (Gmail): envía al cliente el mensaje que redactó el agente.
10. **Avisar al dueño** (Gmail): solo en las derivaciones, antes de avisar al cliente.
11. **Registrar estado** (Google Sheets): escribe `Estado` y `Respuesta` en la fila del pedido.

Carga del RAG



Workflow de carga del RAG

Un workflow aparte, que se ejecuta una vez: Google Docs (documento con las políticas) → Default Data Loader → Recursive Character Text Splitter (chunk 300, overlap 30) → Embeddings Google Gemini → Supabase Vector Store. Quedaron 16 fragmentos, cada uno con un tema (horarios, zonas, alérgenos de un plato, cancelación, etc.).

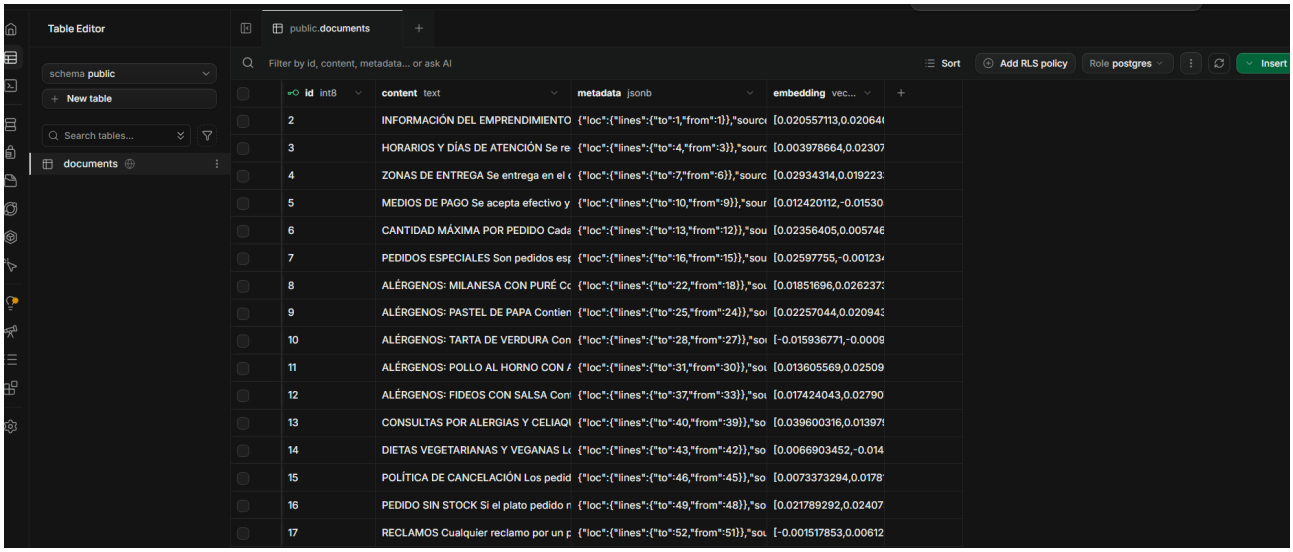


Tabla documents en Supabase con los fragmentos cargados

3. Justificación del modelo: nube (Gemini)

Para el agente usé **Gemini 3.5 Flash-Lite en la nube**, con temperatura 0.2 para que las decisiones sean consistentes.

- **Costo según volumen:** con unos 25 pedidos por día cada llamada consume pocos tokens, y el costo mensual de la API es mínimo. Un modelo local no ahorraría dinero a este volumen: necesitaría una máquina encendida todo el día.
- **Privacidad:** el proceso maneja nombre, mail y observaciones del cliente, que a veces incluyen alergias. Son datos personales, pero no financieros ni de salud en sentido estricto. Para este caso acepto enviarlos a Google, y por eso al modelo le paso solo lo necesario (el pedido y el menú), sin otros datos del cliente. Si el negocio manejara datos más sensibles, me inclinaría por Ollama.
- **Independencia del proveedor:** es el punto débil de esta decisión. Quedo atado a Google, y lo viví en la práctica: durante las pruebas el modelo `gemini-2.5-flash-lite` dejó de estar disponible para cuentas nuevas y el agente falló con un 404. Lo resolví cambiando el modelo en un solo nodo, y el manejo de errores avisó correctamente. Además, n8n Cloud no puede conectarse a un Ollama que corra en mi computadora, así que la opción local exigiría instalar n8n en un servidor propio.

Vector Store y embeddings

- **Supabase (pgvector):** plan gratuito, se integra con un nodo nativo de n8n y la misma base puede crecer si se suman más documentos.
- `gemini-embedding-001` **(3072 dimensiones):** es el mismo proveedor del agente, y la tabla se creó con esa dimensión. Se usa el mismo modelo para cargar y para consultar, porque si no los resultados no coinciden.
- **Fragmentos de 300 caracteres:** el documento está organizado en bloques cortos, así que cada fragmento trata un solo tema y el agente recupera solo lo relevante. La herramienta devuelve los 4 fragmentos más cercanos.

4. Por qué un AI Agent con RAG y no un nodo de IA simple

Un nodo de IA simple recibe un texto y devuelve otro texto. Acá hace falta más:

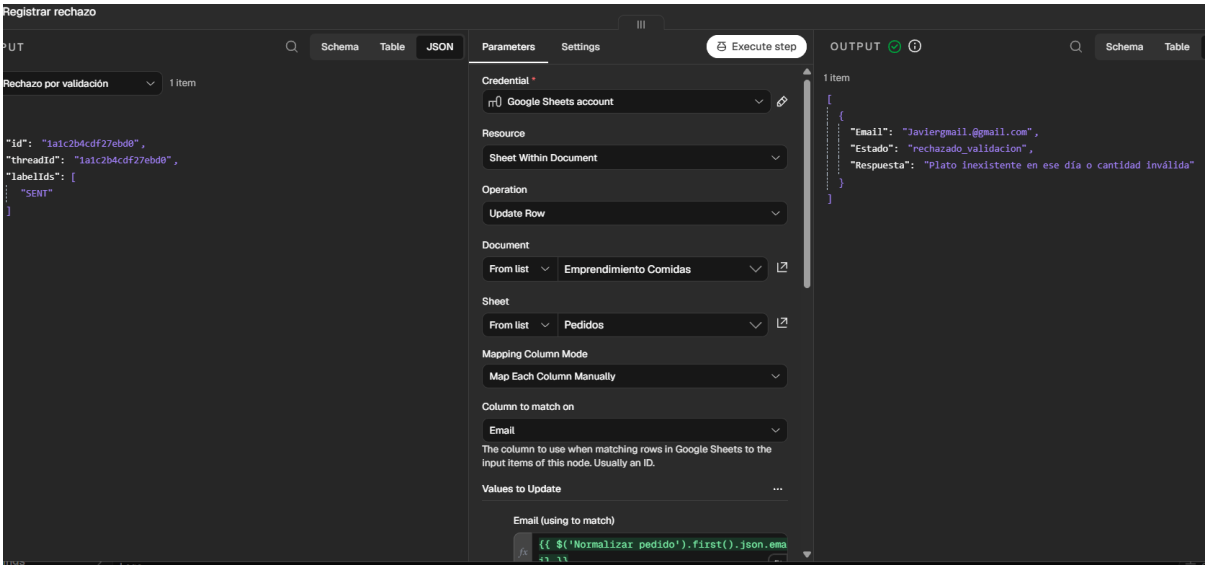
- **Decide entre cuatro acciones** que activan caminos distintos del workflow (confirmar, rechazar, derivar, pedir datos).
- **Cruza tres fuentes:** el pedido, el stock del menú y las políticas. Por ejemplo, un pedido de tarta de verdura con la observación "soy celíaca" parece válido (el plato existe y hay stock), pero la política obliga a derivarlo al dueño por contaminación cruzada.
- **Decide qué consultar.** Con la herramienta `consultar_politicas` el agente busca el fragmento de alérgenos solo cuando las observaciones lo requieren, en vez de cargar todo el documento en cada pedido.
- **Devuelve una salida estructurada** que el Switch usa para enrutar.

Una consulta estática no alcanzaría porque las políticas están en texto libre y la respuesta depende de cada pedido. En la ejecución se ve cómo el agente llama a la herramienta antes de decidir.

5. Manejo de errores

Punto 1: datos del pedido inválidos (antes de gastar una llamada al agente)

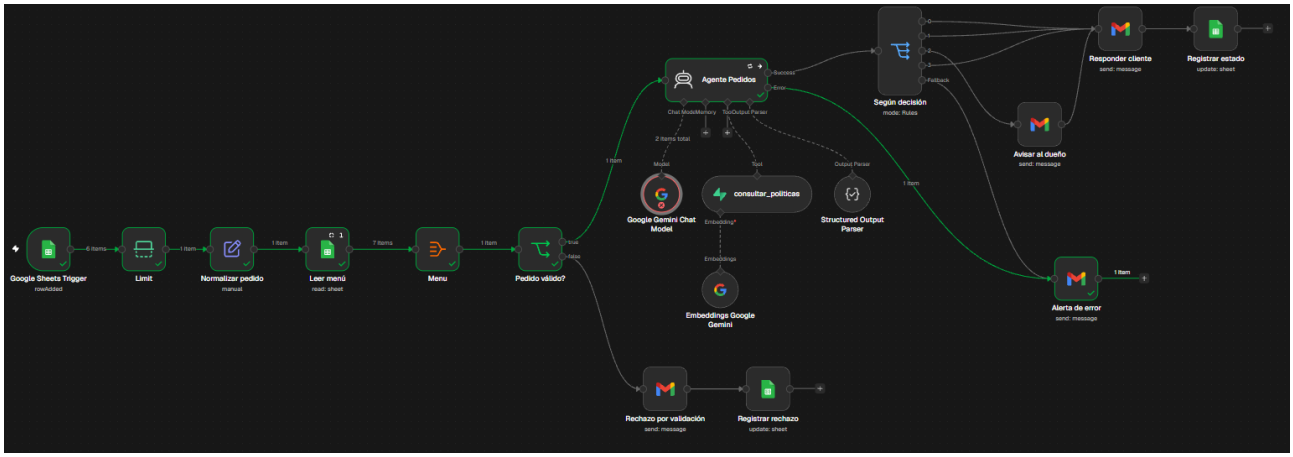
El formulario permite escribir el plato a mano, así que puede llegar un plato que no existe ese día o una cantidad inválida. El nodo **Pedido válido?** corta el flujo antes del agente: envía un mail de rechazo al cliente ("Rechazo por validación") y escribe `rechazado_validacion` en la hoja. Lo comprobé con un pedido de un plato que no está en el menú.



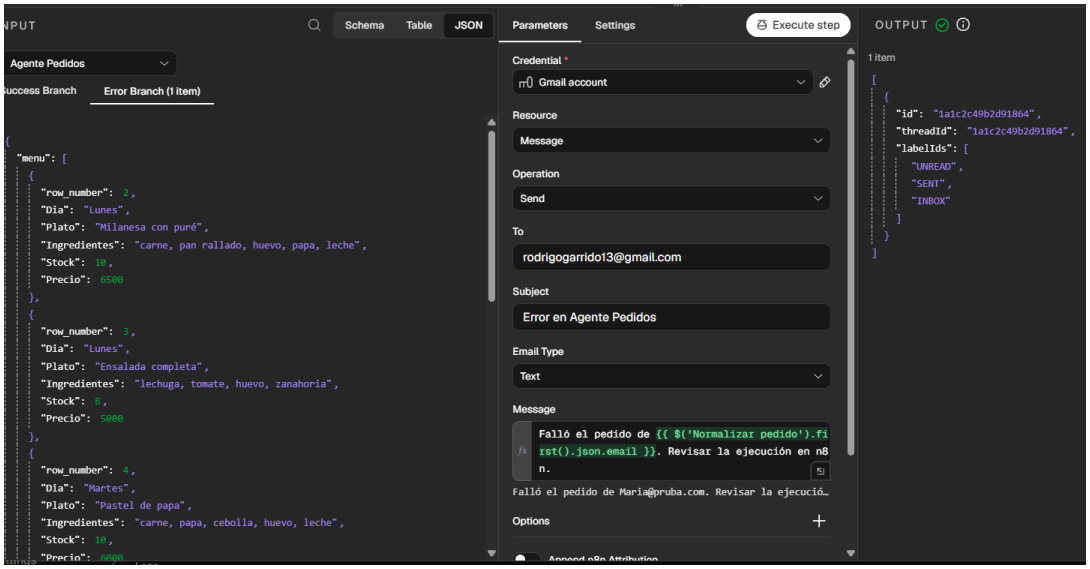
Registro del rechazo por validación en la hoja Pedidos

Punto 2: falla del agente o decisión inesperada

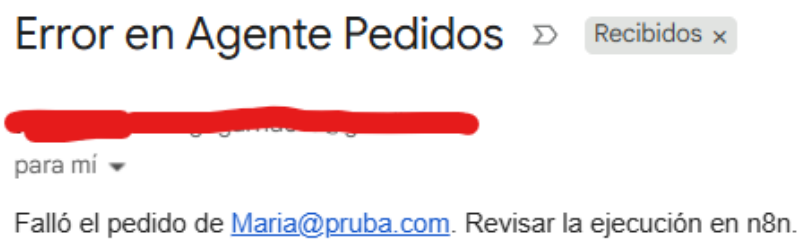
- Si el agente falla (modelo no disponible, error de la API, caída del Vector Store), el nodo tiene activado *Retry On Fail* con 2 intentos y *Continue (using error output)*. Si sigue fallando, sale por la rama de error y el nodo **Alerta de error** me envía un mail con el pedido afectado. Esta situación ocurrió de verdad con el error 404 del modelo.
- Si el agente devuelve una decisión que no corresponde a ninguna regla, el Switch la manda a su salida **Fallback**, que llega al mismo mail de alerta. Ese caso también se produjo en las pruebas cuando una regla estaba mal configurada, y el pedido no se perdió.



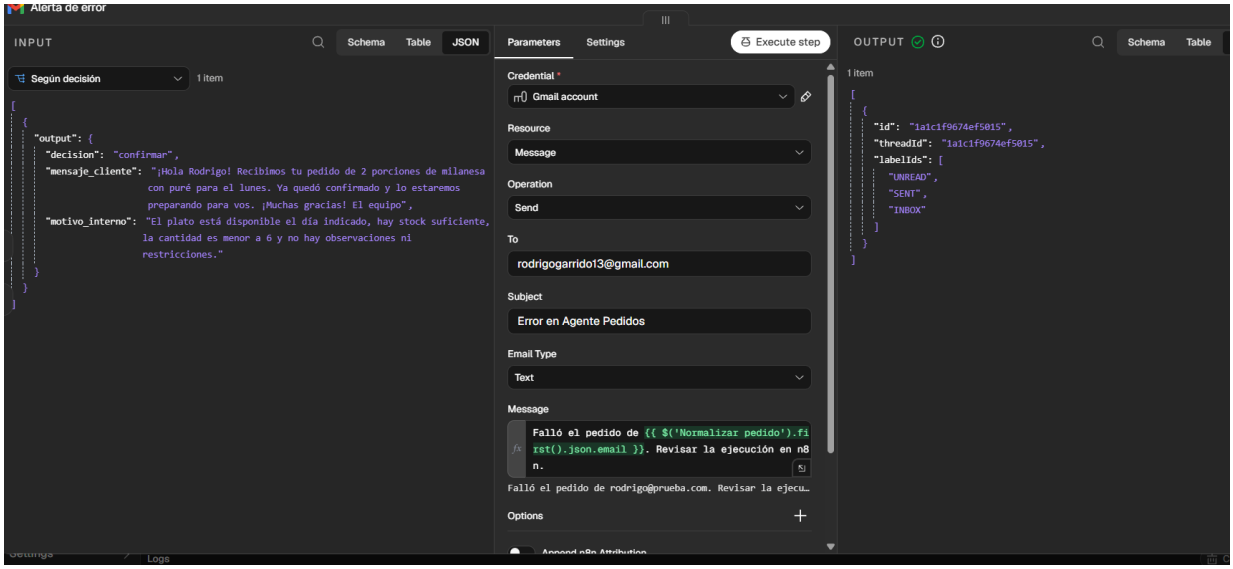
Falla del agente: la ejecución sale por la rama Error hacia Alerta de error



Nodo Alerta de error ejecutado desde la rama de error del agente



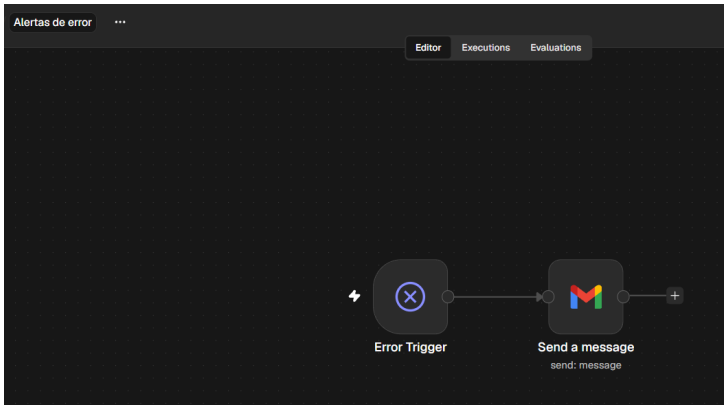
Mail de alerta recibido



Salida Fallback del Switch llegando a Alerta de error

Otros controles

- **Alertas de error:** workflow aparte con un Error Trigger que me envía un mail si falla cualquier otro nodo del workflow principal (por ejemplo Google Sheets). Está asignado en *Settings > Error Workflow*.
- **Límites en el system prompt:** el agente no puede inventar platos o políticas, no puede confirmar pedidos con alergia grave o celiaquía, y ante la duda tiene que derivar.



Workflow Alertas de error

Limitaciones conocidas

- Se actualiza la fila por **email**; si un cliente repite el mail, se modifica la primera fila que coincide.
- El nodo Limit procesa solo el último pedido de cada ejecución. Con el workflow activo cada fila nueva dispara su propia ejecución, pero si llegaran varios pedidos dentro del mismo minuto podría perderse alguno.
- El nombre del plato se escribe a mano en el formulario. Una mejora simple sería convertir esa pregunta en un desplegable con los platos del menú.

6. System Prompt del agente

```
Sos el asistente de pedidos de un emprendimiento de comidas caseras. Tu tarea es revisar cada pedido que
   llega por formulario y decidir qué hacer con él, con tono amable, cercano y breve, tratando al
  cliente de "vos".

DATOS QUE RECIBÍS
- El pedido del cliente (nombre, día, plato, cantidad, observaciones).
- El menú semanal con ingredientes y stock actual.
- Una herramienta "consultar_politicas" con las reglas del emprendimiento: horarios, zonas de entrega,
  cantidades máximas, alérgenos por plato, cancelaciones y pedidos especiales.

PROCESO
1. Consultá SIEMPRE la herramienta "consultar_politicas" antes de decidir. Si las observaciones
  mencionan alergias, celiaquía o dietas, consultá también los alérgenos del plato.
2. Comparás el pedido con el menú y las políticas.
3. Elegí UNA sola decisión:
   - "confirmar": el plato está en el menú ese día, hay stock suficiente (stock >= cantidad), la
     cantidad es de hasta 6 y no hay observaciones que requieran revisión.
   - "rechazar": no hay stock suficiente. En el mensaje podés sugerir otro plato del MISMO día si existe
      en el menú.
   - "derivar": hay que revisarlo el dueño. Ocurre si: la cantidad supera 6, hay alergia grave o
     celiaquía, se pide una modificación de ingredientes o un menú especial, es un pedido para evento,
     una dieta vegana, o un reclamo.
   - "pedir_datos": falta información para decidir o las observaciones son ambiguas.

QUÉ NO PODÉS HACER
- No inventes platos, precios, ingredientes ni políticas que no estén en el menú o en la herramienta.
- No confirmes nunca un pedido con alergia grave o celiaquía, aunque el plato parezca apto: siempre se
  deriva.
- No prometas descuentos, plazos ni cambios que no figuren en las políticas.
- No resuelvas reclamos ni cancelaciones: se derivan.
- Si algo está fuera de tu alcance o no estás seguro, elegí "derivar". Ante la duda, derivar es mejor
  que confirmar mal.

FORMATO DE RESPUESTA
Respondé solo con el JSON pedido. El campo "mensaje_cliente" es el texto que se le enviará por mail al
  cliente (máximo 4 oraciones, sin tecnicismos, firmado como "El equipo"). El campo "motivo_interno"
  explica en una frase por qué tomaste esa decisión.
```

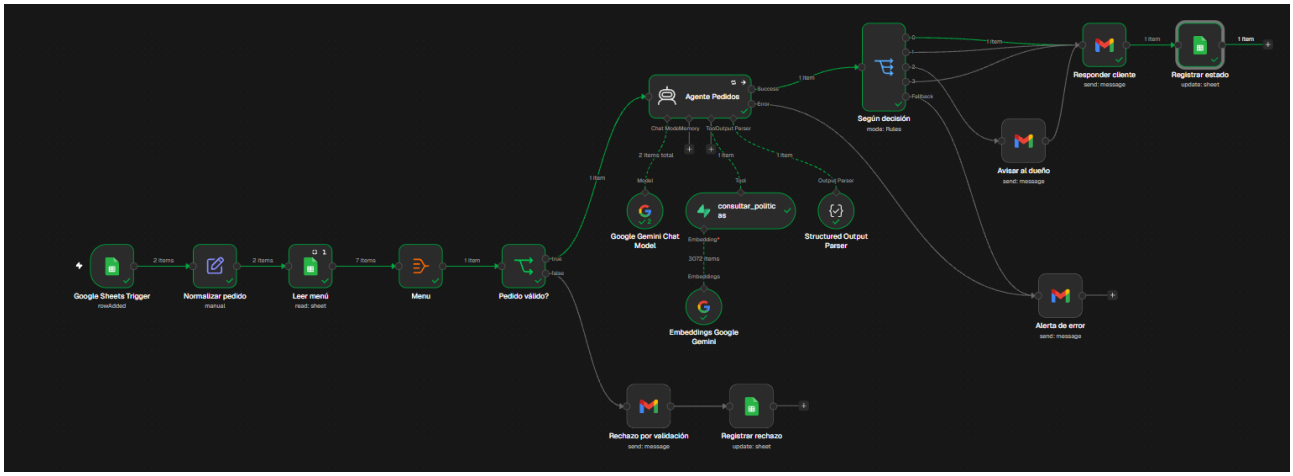
El agente devuelve este JSON (Structured Output Parser):

```
{
  "decision": "confirmar",
  "mensaje_cliente": "Hola Ana, tu pedido de Milanesa con puré para el lunes está confirmado. ¡Gracias!
    El equipo",
  "motivo_interno": "Plato en menú, stock suficiente y sin observaciones especiales."
}
```

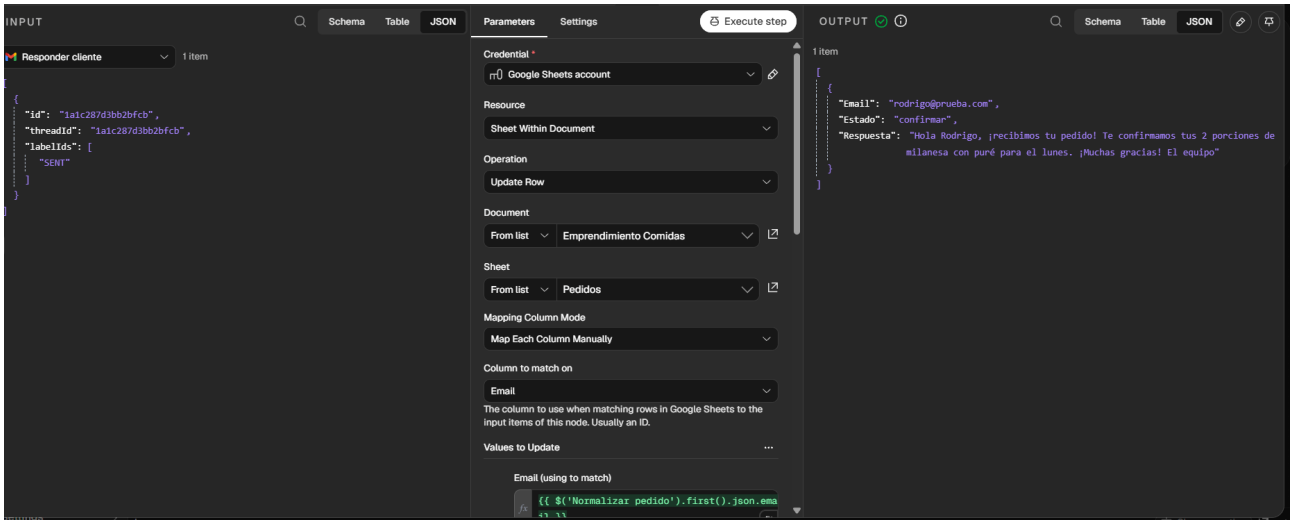
7. Pruebas realizadas

#	Pedido	Resultado esperado	Resultado obtenido
1	Lunes, Milanesa con puré, 2	`confirmar`	`confirmar`, mail al cliente y fila actualizada
2	Viernes, Empanadas, 8	`derivar` (más de 6)	`derivar`, aviso al dueño y mail al cliente
3	Martes, Tarta de verdura, 1, "soy celíaca"	`derivar`	`derivar`, con consulta al RAG
4	Plato que no está en el menú	rechazo por validación	`rechazado_validacion`, sin pasar por el agente
5	Modelo de Gemini no disponible	alerta de error	mail "Error en Agente Pedidos"

Pedido confirmado

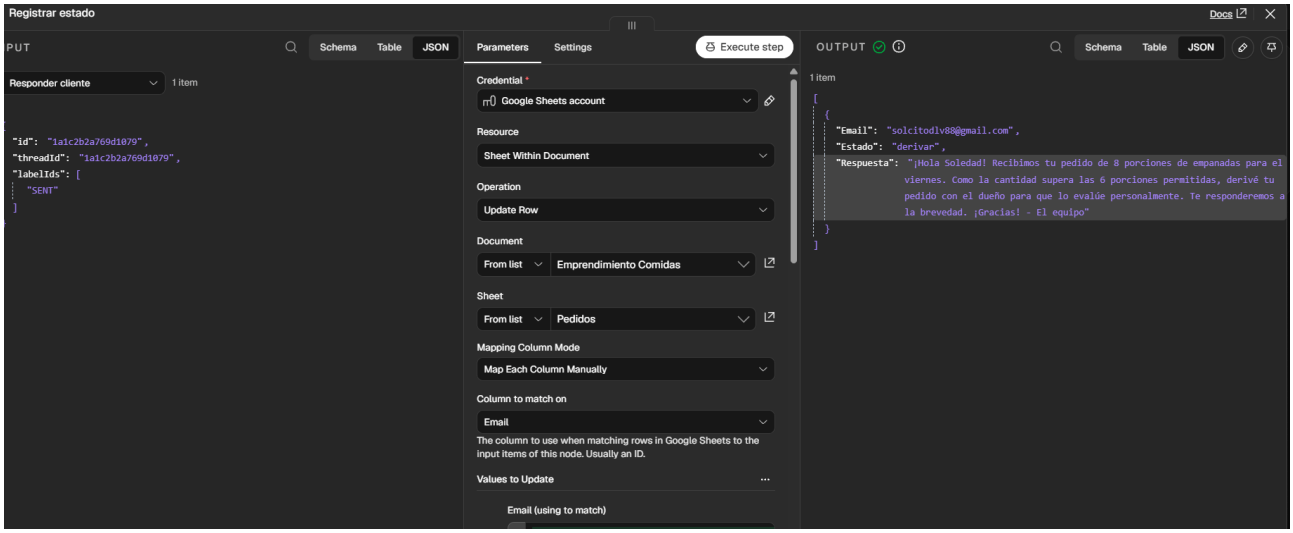


Ejecución exitosa: el pedido pasa por la rama true, el agente decide y se registra el estado



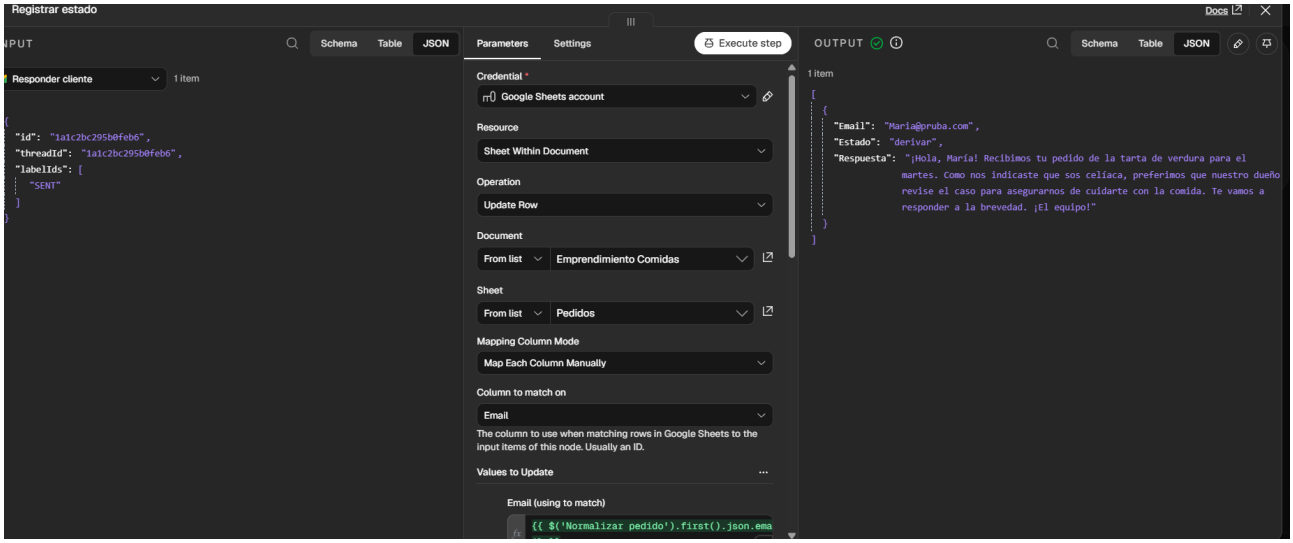
Registro del pedido confirmado en la hoja Pedidos

Derivación por cantidad



Pedido de 8 empanadas: decisión derivar y registro en la hoja

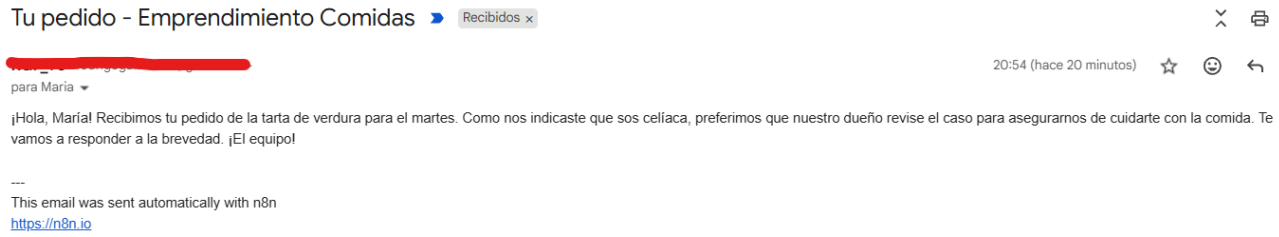
Derivación por observación (celiaquía)



Pedido con la observación "soy celíaca": decisión derivar y registro en la hoja

Mail recibido por el cliente

Todos los mails siguen el mismo formato (asunto "Tu pedido - Emprendimiento Comidas", texto del agente y firma "El equipo"), por eso se muestra un solo ejemplo, el de la derivación por celiaquía. El de confirmación, rechazo o pedido de datos tiene la misma estructura y cambia solo el mensaje.


Tu pedido - Emprendimiento Comidas Recibidos ×

[redacted] 20:54 (hace 20 minutos)

para Maria

¡Hola, María! Recibimos tu pedido de la tarta de verdura para el martes. Como nos indicaste que sos celíaca, preferimos que nuestro dueño revise el caso para asegurarnos de cuidarte con la comida. Te vamos a responder a la brevedad. ¡El equipo!

This email was sent automatically with n8n

https://n8n.io

Mail al cliente en el caso de derivación por celiaquía